

Real-Time Indian Sign Language Interpretation by Deep Neural Network and MediaPipe Framework

Devesh Singh*

School of Computer Science and Engineering
Shri Mata Vaishno Devi University
Katra, Jammu, India
singhdevesh582@gmail.com

Manoj Kumar Gupta

School of Computer Science and Engineering
Shri Mata Vaishno Devi University
Katra, Jammu, India
manoj.gupta@smvdu.ac.in

Abstract—In a world where spoken language dominates, people who are deaf or hard of hearing often encounter communication barriers. To overcome these barriers, they have developed sign language—a visual form of communication using hand signs and movements. Sign language uses a combination of hand signs and movements to communicate. India has developed its own separate sign language called the Indian sign language (ISL). It currently has about 18 million day-to-day users. The Indian Sign language is recognized as one of the modern Indian languages. Therefore, it is necessary to develop tools that can interpret these sign languages without the need of human interpreters.

This work aims to establish a ML driven solution to interpret the Indian sign language to text using MediaPipe framework to detect the ISL hand signs using webcam through OpenCV and translate them in real-time. The MediaPipe framework was used to detect and extract about 36200 (about 1000 data points each for 0-9 and A-Z) data points which were used to train feed forward deep neural network resulting in a 99% accuracy. Enabling translation in real-time using OpenCV in an effective manner. This method therefore can effectively be tried for other sign languages and enable an inclusive and better environment for deaf and mute people. A similar work was done by Jyotishman bora [13] on Assamese Sign Language for 9 static, single handed, different poses here we have extended the scope of their work to entire Indian Sign Language with 36 different pose recognitions. This work supports both one-handed and two-handed static ISL gestures in contrast to the previous work and achieves an average inference speed of 30 FPS on an AMD Ryzen 7 5000 series system. This real-time performance is enabled through an asynchronous multi-threaded processing pipeline combined with frame-skipping and stationarity-based temporal filtering, which reduces redundant inference during gesture transitions. This confirms its suitability for real-time applications.

Index Terms—Sign Language Interpretation; Indian Sign Language (ISL); Gesture-Based Recognition; MediaPipe Framework; Deep Neural Networks

I. INTRODUCTION

WHO estimates about 430 million deaf-mute individuals worldwide, including roughly 70 million in India (6.3 % of its population). Sign language—a visual system of hand gestures—varies by region: American Sign Language (ASL) and Indian Sign Language (ISL) are the most widely used, with some 18 million Indians relying on ISL despite only 250–350 certified interpreters in the country.

Sign Language Recognition (SLR) systems fall into two categories:

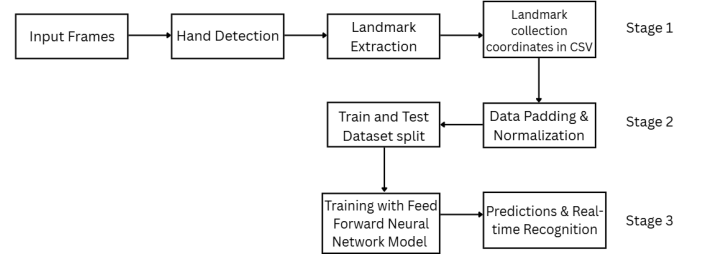


Fig. 1: Overview Diagram of the Entire System Workflow

Glove-based, which use sensor-equipped gloves to capture hand motion and orientation. [1] [2] [3]

Vision-based, which employ cameras and computer-vision techniques (edge detection, skin segmentation, gesture and hand-shape analysis). [4] [5]

Vision-based methods often demand high computational power and suffer from unstable hand tracking on low-end devices.

We present a lightweight, real-time ISL recognizer that combines Google's MediaPipe hand-tracking with a feed-forward neural network. MediaPipe detects 21 hand landmarks per frame; their normalized coordinates train the network. Integrated with OpenCV for live capture, our system recognizes letters A–Z and digits 0–9 with low latency and high accuracy. Figure 1 outlines the pipeline. We detail its design, implementation, and performance evaluation.

II. LITERATURE REVIEW

Das et al. [5] developed an ASL static-gesture recognizer using an Inception V3 CNN trained on 24 classes (A–Z, excluding J) with roughly 100 RGB-captured images per class. Their network achieved over 90 % average validation accuracy, peaking at 98 %, indicating Inception V3's effectiveness for a dataset of cropped static sign detection.

Sahoo [7] evaluated static ISL digit recognition (0–9) using 5,000 RGB images (500 per class). He compared Naïve Bayes and k-Nearest Neighbors classifiers, with k-NN achieving 98.36 % average accuracy versus 97.79 % for Naïve Bayes.

Ansari and Harit [4] used Microsoft Kinect to capture 5,041 RGB-D images covering 140 static ISL gestures, then applied k-means clustering for classification—achieving 100 % accuracy on three signs, 90 % on thirteen, and an overall accuracy of 90.68 % on a total of 16 alphabetic characters that were : A, B, D, E, F, G, H, K, P, R, T, U, W, X, Y, and Z

Rekha et al. [8] utilized a skin color-based segmentation approach for hand detection. They extracted features based on edge orientation and texture, which were then used to train a multiclass Support Vector Machine (SVM) 23 static and 3 dynamic ISL signs, achieving 86.3 % accuracy—but the processing was too slow for practical deployment.

Bhuyan et al. [9] achieved a recognition rate exceeding 90% on a dataset of 400 images covering 8 Indian Sign Language gestures. Their method first employed skin color segmentation for hand detection, followed by a nearest neighbor classification algorithm.

Pugeault and Bowden [10] trained a system on a collection of 48,000 3D depth images to create a tool capable of recognizing American Sign Language alphabets in real time, spanning 24 classes, collected via a Kinect sensor. Their approach utilized Gabor filters to extract features and multi-class random forests for classification, resulting in highly accurate recognition of static hand gestures.

Keskin et al. [11] implemented a part-based object recognition method to identify American Sign Language digits. Their dataset was composed of 30,000 3D depth images captured by a Kinect sensor, which covered ten distinct classes and resulted in a 99% accuracy rate.

Halder and Tayade [12] achieved an average accuracy of approximately 99% in real-time hand sign detection. Their method involved using the MediaPipe framework to extract multi-hand landmarks, which were then classified using a Support Vector Machine (SVM).

Bora et al. [13] employed a comparable MediaPipe-feed-forward network pipeline to distinguish nine Assamese Sign Language symbols using 2,094 single-handed static gesture samples. While their model achieved 99% accuracy, it was limited to a smaller one handed gesture set that limited its scope for further development. In contrast, our system supports static gestures performed with either one or both hands across 36 ISL classes, significantly increasing the complexity and usability of the system.

III. DATA COLLECTION PROCEDURE

A. Simulation Environment

OpenCV library was used to interact with the webcam and capture the images from a live video feed and each frame from the feed was used for detecting landmark points and logged in a .csv file for training. The deep learning was implemented on a jupyter notebook environment. The live input feed is shown in Figure 2.

B. Hand Landmark Detection Using MediaPipe

MediaPipe is a lightweight, open-source framework (released by Google in 2019) for building real-time audio and



Fig. 2: Live video Input Feed

video pipelines—even on low-power devices like Android phones or IoT gadgets. It consists of a C++, Java, and Objective-C core plus two layers: a “Calculator” graph API for defining data flows and a set of prebuilt solutions (16 TensorFlow/TensorFlow Lite models) for common tasks. In our system, we employ MediaPipe’s hand-tracking solution, which first uses a palm detector to crop the most stable region of the hand, then applies a landmark regressor to output 21 three-dimensional joint coordinates per hand (42 points total). Trained on 30,000 annotated images, this two-step pipeline accurately localizes hand landmarks in each frame—even when parts of the hand extend beyond the camera view.

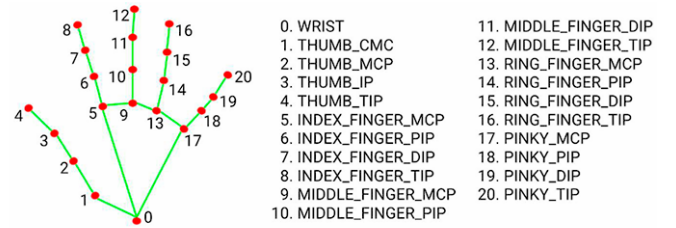


Fig. 3: The 21 landmark points taken by Mediapipe on one hand

Each webcam frame is processed by MediaPipe’s hand-tracking model—with a visual overlay confirming detected landmarks. The model yields 3D (x, y, z) coordinates, but we discard z and normalize the remaining x–y pairs, logging them to a CSV file for training.

C. Normalization and Padding

MediaPipe outputs landmarks in pixel coordinates, so identical gestures at different frame positions yield inconsistent values. To normalize, we reset the wrist landmark (index 0) to (0,0) by subtracting its x–y coordinates from all other points. We then divide each offset by the largest absolute value across all coordinates, scaling them into [-1,1]. These normalized x–y pairs are streamed and logged to a CSV in real time. Figure 5 illustrates the normalization pipeline, and Figure 6 shows the resulting landmark distributions.

media. The dataset also includes variations in hand shape and size, backgrounds ranging from plain to cluttered, multiple camera angles, and diverse lighting conditions (including indoor fluorescent and natural daylight). This diversity in the data improves the model's ability to perform well on new data and makes it more resilient in practical applications.

IV. MODEL TRAINING USING A CUSTOM FEEDFORWARD NEURAL NETWORK

We trained a feed-forward network with an input layer, two hidden dense layers (with ReLU activations) separated by dropout, and a Softmax output layer (see Table 1). Dense layers compute the linear mappings, while dropout randomly zeros connections to curb overfitting. To prepare the model for training, it was configured to use the Adam optimizer, the sparse categorical cross-entropy loss function, and accuracy as the primary performance metric. Early stopping terminated training after 135 of the allotted 1000 epochs once loss and accuracy stabilized. It achieved 99.27 % training accuracy with a final loss of 0.0481.

TABLE I: Configuration of the used Feedforward Neural Network

Layer (type)	Output Shape	Number of Parametrs
dropout (Dropout)	(None, 84)	0
dense (Dense)	(None, 64)	5,440
dropout_1 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 36)	1,188
Total params:		8,708 (34.02 KB)
Trainable params:		8,708 (34.02 KB)
Non-trainable params:		0 (0.00 B)

The study by Bora et al. [13] used 42 input parameter feed forward neural network with 1,180 total trainable parameters that is only able to handle single handed gestures whereas we used a 84 input parameter with padding so that we can use both one handed and two handed gestures therby increasing the total trainable parameters to 8,708.

V. RESULTS AND ANALYSIS

A. Quantitative analysis of the model

We applied scikit-learn's 'classification_report' and 'confusion_matrix' on our test set to obtain accuracy, precision, recall, F1 score, and support. Accuracy (Eq. 1) is the fraction of correct predictions over all samples. Precision (Eq. 2)—true positives divided by predicted positives—is crucial when false positives carry a high cost. Recall (Eq. 3)—true positives over actual positives—matters most if false negatives are costly. Finally, the F1 score (Eq. 4) combines precision and recall as their harmonic mean, providing a single balanced metric.

$$\text{Accuracy (A)} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

$$\text{Precision (P)} = \frac{TP}{TP + FP} \quad (2)$$

$$\text{Recall (R)} = \frac{TP}{TP + FN} \quad (3)$$

$$\text{F1 Score (F)} = \frac{2 \cdot P \cdot R}{P + R} \quad (4)$$

Table 2 contains a detailed classification report for the feedforward neural network model, which performed with an average accuracy of 99%. The abbreviations TP (True Positive), TN (True Negative), FP (False Positive), and FN (False Negative) are used in the corresponding equations (1) through (4).

TABLE II: Feedforward Model Classification Report

Class	Precision	Recall	F1-Score	Support
0	1.00	1.00	1.00	253
1	1.00	0.99	0.99	232
2	0.99	1.00	0.99	243
3	1.00	0.99	0.99	267
4	1.00	0.99	0.99	230
5	1.00	1.00	1.00	234
6	0.98	1.00	0.99	242
7	1.00	1.00	1.00	255
8	0.99	1.00	1.00	261
9	1.00	1.00	1.00	254
10	1.00	0.98	0.99	260
11	1.00	0.98	0.99	283
12	0.96	0.99	0.97	255
13	1.00	0.98	0.99	219
14	0.99	0.99	0.99	278
15	1.00	1.00	1.00	235
16	0.98	0.97	0.97	241
17	1.00	0.97	0.99	226
18	0.97	0.98	0.98	258
19	0.99	1.00	0.99	261
20	1.00	1.00	1.00	269
21	0.99	1.00	0.99	263
22	1.00	1.00	1.00	270
23	0.99	1.00	0.99	288
24	1.00	1.00	1.00	261
25	1.00	1.00	1.00	229
26	1.00	1.00	1.00	266
27	0.98	1.00	0.99	258
28	0.99	0.98	0.99	234
29	1.00	1.00	1.00	246
30	1.00	0.99	0.99	228
31	1.00	0.98	0.99	232
32	1.00	1.00	1.00	238
33	1.00	1.00	1.00	263
34	1.00	1.00	1.00	249
35	0.98	1.00	0.99	269
Accuracy	0.99			9050
Macro avg	0.99			9050
Weighted avg	0.99			9050

We leveraged scikit-learn's 'confusion_matrix' during live inference to tabulate true versus predicted labels per class. This matrix not only quantifies overall recognition accuracy but also highlights systematic misclassifications. The resulting plot is presented in Figure 9.

B. Real-Time Recognition

Figure 10 illustrates the model's live application with several snapshots showing its implementation using the OpenCV library to access the webcam and integrate it with the trained model. This setup effectively recognizes live Indian Sign Language gestures with high accuracy.

During live testing on an AMD Ryzen 7 5000 series laptop with 16 GB RAM, the system achieved an average inference speed of approximately 30 FPS using OpenCV and MediaPipe. To maintain this real-time performance, the system employs an asynchronous multi-threaded architecture that decouples frame capture from inference using a bounded producer-consumer queue, ensuring that only the most recent frames are processed when inference lags behind capture.

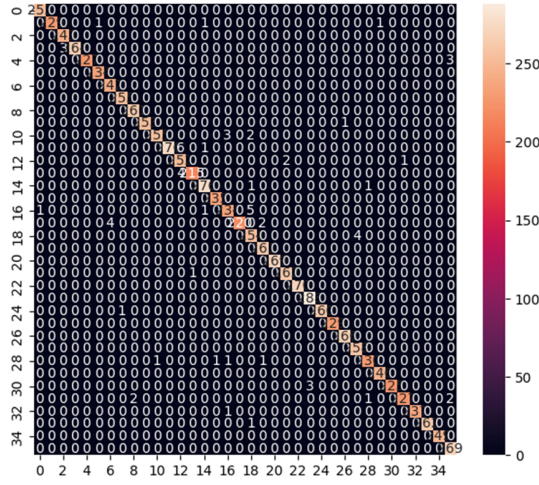


Fig. 9: Confusion Matrix of the Feed forward neural network used

Additionally, a stationarity check prevents redundant processing by running the classifier only when the hand coordinates stabilize within a defined pixel threshold, significantly reducing computational overhead during transition movements. This confirms the framework’s capability for real-time ISL interpretation in standard computing environments.

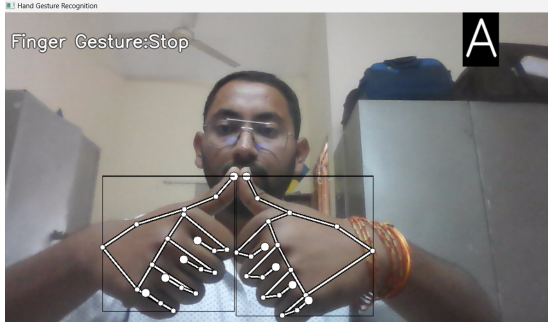


Fig. 10: Real-time implementation of the model

C. Comparative Analysis

A comparison of the results from this work with other studies is provided in Table 3 and illustrated in Figure 11. Here, the model is also compared to the traditional CNN approaches to image recognition. In addition to high accuracy, our model is well-suited for low-power environments. With just 8,708 trainable parameters (34 KB), it is significantly smaller than typical CNNs. MediaPipe’s pretrained hand-landmark detector handles localization and feature extraction, outputting 42-point skeletal data per hand. This allows the feed-forward network to focus solely on classification, reducing complexity and inference time. The decoupled design enables real-time performance on devices like Raspberry Pi using TensorFlow Lite. Future work will include device-specific benchmarks to validate latency.

A direct comparison with Bora et al. (2023) is shown in Table 4. Our model improves on their work by handling accommodating both single- and double-handed gestures, which resulted in a fourfold increase in the class count from 9 to 36, while maintaining high accuracy and supporting real-time inference.

TABLE III: Comparison of various Sign Language Recognition studies

Refs.	Sign Language	Main Techniques	Accuracy
[14]	American Sign Language (ASL)	CNN	96.30%
[15]	Various	MobileNet, CNN, LSTM, MRFO	99.28%
[16]	Pakistan Sign Language (PSL)	CNN, SIFT	98.74%
[17]	Chinese Sign Language	CNN, Stretchable strain sensors, IMUs	95.85% (W), 84% (S)
[18]	American and British Sign Language (ASL & BSL)	CNN, MediaPipe, LSTM, DTW-KNN	97.24%, 87.4%, 83.7%
[19]	Indian Sign Language (ISL)	3D CNN	88.24%
This Work	Indian Sign Language	MediaPipe, Feed-forward neural network	99%

TABLE IV: Comparison with the previous work

Study	Gesture Type	No. of Classes	Accuracy	FPS / Platform
Bora et al. (2023) [13]	One-handed Static	9	99%	Not Reported
This Work	One- and Two-handed Static	36	99%	30 FPS (Ryzen 7, 16 GB RAM)

VI. CONCLUSION

This study currently supports only static hand gestures; dynamic sign sequences remain future work, likely requiring temporal models such as LSTM or TCN. The system may also experience reduced accuracy under extreme low-light or heavy occlusion.

On ethics, all gesture data were self-recorded—no other individuals participated, and no raw images or video were stored. Only anonymized (x, y) landmark vectors were logged locally, with no external data transmission. Consequently, this research involves no personally identifiable biometric data, aligning fully with GDPR and India’s DPDP Act (2023). Prior to any public deployment, we will develop explicit user-consent and data-handling guidelines to ensure continued compliance and user privacy.

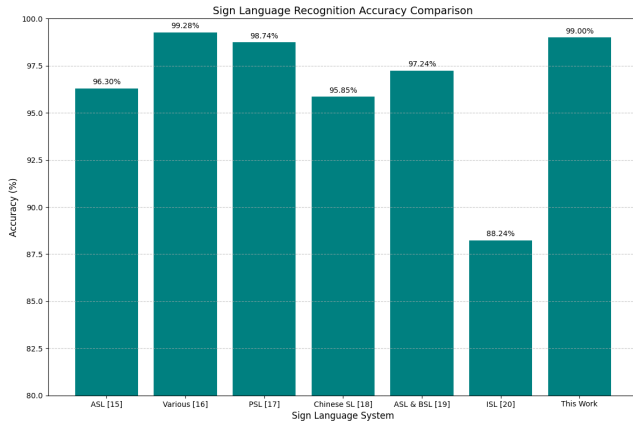


Fig. 11: Comparison plot of various Sign Language Recognition Studies

VII. LIMITATIONS AND ETHICAL CONSIDERATIONS

This study currently supports only static hand gestures; dynamic sign sequences remain future work, likely requiring temporal models such as LSTM or TCN. The system may also experience reduced accuracy under extreme low-light or heavy occlusion.

On ethics, all gesture data were self-recorded—no other individuals participated, and no raw images or video were stored. Only anonymized (x, y) landmark vectors were logged locally, with no external data transmission. Consequently, this research involves no personally identifiable biometric data, aligning fully with GDPR and India’s DPDP Act (2023). Prior to any public deployment, we will develop explicit user-consent and data-handling guidelines to ensure continued compliance and user privacy.

VIII. FUTURE WORK

MediaPipe reliably supplies the low-level hand and face landmarks, but interpreting full sign language—where temporal patterns, facial expressions, and grammar convey meaning—requires a layered interpretation engine. We propose the following research directions to build such an engine:

Temporal modeling: replace or augment the current feed-forward classifier with sequence models (LSTM/GRU), Temporal Convolutional Networks (TCN), or transformer-based encoders to capture motion and sign transitions for dynamic gestures and fingerspelling sequences.

Multimodal fusion: combine hand landmarks with facial landmark streams (eyebrows, lip motion), upper-body pose, and optionally audio/text context. Explore early, late, and intermediate fusion strategies and attention mechanisms to weight modalities adaptively.

Structured representations: represent landmarks as graphs and apply Graph Neural Networks or Temporal Graph Networks to better model inter-landmark relationships and two-hand interactions.

Pretraining & data strategies: use self-supervised or contrastive pretraining on large unlabeled landmark sequences,

augment datasets (synthetic variations, domain randomization), and investigate domain adaptation to generalize across signers, cameras, and lighting.

End-to-end translation: study seq2seq and transformer architectures that map landmark sequences directly to natural language (text/speech), including integration with language models for grammatical reconstruction.

REFERENCES

- [1] A. Z. Shukor *et al.*, “A new data glove approach for Malaysian sign language detection,” *Procedia Computer Science*, vol. 76, pp. 60–67, 2015.
- [2] M. Mohandes, M. Deriche, and J. Liu, “Image-based and sensor-based approaches to Arabic sign language recognition,” *IEEE Trans. Human-Machine Syst.*, vol. 44, pp. 551–557, 2014.
- [3] N. M. Kakoty and M. D. Sharma, “Recognition of sign language alphabets and numbers based on hand kinematics using a data glove,” *Procedia Computer Science*, vol. 133, pp. 55–62, 2018.
- [4] Z. A. Ansari and G. Harit, “Nearest neighbour classification of Indian sign language gestures using Kinect camera,” *Sādhanā*, vol. 41, pp. 161–182, 2016.
- [5] A. Das, S. Gawde, K. Suratwala, and D. Kalbande, “Sign language recognition using deep learning on custom processed static gesture images,” in *Proc. Int. Conf. Smart City and Emerging Technology (ICSCET)*, 2018.
- [6] C. Lugaresi *et al.*, “MediaPipe: A framework for building perception pipelines,” *arXiv preprint arXiv:1906.08172*, 2019.
- [7] A. K. Sahoo, “Indian sign language recognition using machine learning techniques,” *Macromol. Symp.*, 2021.
- [8] J. Rekha, J. Bhattacharya, and S. Majumder, “Shape, texture and local movement hand gesture features for Indian sign language recognition,” in *Proc. 3rd Int. Conf. Trendz in Information Sciences & Computing (TISC)*, 2011.
- [9] M. K. Bhuyan, M. K. Kar, and D. R. Neog, “Hand pose identification from monocular image for sign language recognition,” in *Proc. IEEE Int. Conf. Signal and Image Processing Applications (ICSIPA)*, 2011.
- [10] N. Pugeault and R. Bowden, “Spelling it out: Real-time ASL fingerspelling recognition,” in *Proc. IEEE Int. Conf. Computer Vision Workshops (ICCV Workshops)*, 2011.
- [11] C. Keskin, F. Kırac, Y. E. Kara, and L. Akarun, “Real time hand pose estimation using depth sensors,” in *Consumer Depth Cameras for Computer Vision*, Springer, pp. 119–137, 2013.
- [12] A. Halder and A. Tayad, “Real-time vernacular sign language recognition using MediaPipe and machine learning,” *Int. J. Res. Publ. Rev.*, vol. 2, no. 6, p. 7421, 2021.
- [13] J. Bora, S. Dehingia, A. Boruah, A. A. Chetia, and D. Gogoi, “Real-time Assamese Sign Language Recognition using MediaPipe and Deep Learning,” *Procedia Computer Science*, vol. 218, pp. 1384–1393, 2023.
- [14] Y. Obi, K. S. Claudio, V. M. Budiman, S. Achmad, and A. Kurniawan, “Sign language recognition system for communicating to people with disabilities,” *Procedia Computer Science*, vol. 216, pp. 13–20, 2023. [Online]. Available: <https://doi.org/10.1016/j.procs.2022.12.106>
- [15] H. Alsolai *et al.*, “Automated sign language detection and classification using reptile search algorithm with hybrid deep learning,” *Heliyon*, vol. 10, no. 1, p. e23252, 2024. [Online]. Available: <https://doi.org/10.1016/j.heliyon.2023.e23252>
- [16] S. Arooj, S. Altaf, S. Ahmad, H. Mahmoud, and A. S. N. Mohamed, “Enhancing sign language recognition using CNN and SIFT: A case study on Pakistan sign language,” *J. King Saud Univ. – Comput. Inf. Sci.*, vol. 36, no. 2, p. 101934, 2024. [Online]. Available: <https://doi.org/10.1016/j.jksuci.2024.101934>
- [17] Y. Liu *et al.*, “A wearable system for sign language recognition enabled by a convolutional neural network,” *Nano Energy*, vol. 116, p. 108767, 2023. [Online]. Available: <https://doi.org/10.1016/j.nanoen.2023.108767>
- [18] S. N. Koyineni, G. K. Sai, K. Anvesh, and T. Anjali, “Silent expressions unveiled: Deep learning for British and American sign language detection,” *Procedia Computer Science*, vol. 233, pp. 269–278, 2024. [Online]. Available: <https://doi.org/10.1016/j.procs.2024.03.216>
- [19] D. K. Singh, “3D-CNN based dynamic gesture recognition for Indian sign language modeling,” *Procedia Computer Science*, vol. 189, pp. 76–83, 2021. [Online]. Available: <https://doi.org/10.1016/j.procs.2021.05.071>